

Qbase Network Connectivity & Offline Resilience Audit

1. Network Connectivity Monitoring Engine (NetworkMonitor)

The application delegates network status detection to the NetworkMonitor utility class. It uses Android's ConnectivityManager to monitor active network connections dynamically.

- Reactive Flow: Exposes connection status as a cold Flow<Boolean> (isOnline) using callbackFlow to listen for realtime network updates.
- Validation Rules: Verifies internet accessibility via capabilities:
 - * NET_CAPABILITY_INTERNET: The network has the capability to route traffic to the public internet.
 - * NET_CAPABILITY_VALIDATED: The network has been validated by the Android OS to have actual data connectivity (no captive portals or dead routes).
- Performance Optimization: Uses .conflate() on the Flow to prevent backpressure and avoid rendering redundant UI state updates when a connection experiences rapid flapping.

2. App Access & Network States (AppAccessState)

AppAccessState	Conditions	App Functionality & UI Behavior
RestoringSession	!isSessionChecked	App startup. Displays a loading screen while silently validating existing auth sessions without blocking UI interaction.
OnlineReady	user != null && isOnline	
SignedInOffline	user != null && !isOnline	Full operational mode. Live message streaming, Appwrite cloud synchronization, real-time database subscription updates, and active AI capabilities.
GuestOnline	user == null && isOnline	Offline-resilient mode. Users view cached messages/collections. Group management, reports, and messages are queued locally.
OfflineGuest	user == null && !isOnline	Prompt to sign in. Allows standard user authentication via the Login/Register flow.
		Strictly restricted mode. Prompts that internet is required to establish a user session.

3. Core Capabilities & State Influence

A. End-to-End Encrypted (E2EE) Messaging:

- Online: Messages are encrypted locally via Tink (using public keys fetched from Appwrite user profiles), uploaded to the Appwrite messages collection, and received by peers in real-time. Once fetched, receipts are self-healed (deleted from cloud storage).
- Offline: When a user sends a message while in SignedInOffline, the app intercepts it inside ChatViewModel.sendMessage(), sets status to PENDING, stores it locally in the Room database, and warns the user with 'Offline: Message queued.'

B. Chat & Group Administration (Queued Offline):

- Actions like adding/removing group participants, updating roles, and group creation are intercepted and routed through the Universal Action Queue (UniversalQueueManager). Stored locally in Offline_Actions table.

C. User Profiles & Settings (Queued Offline):

- Profile updates (display name, bio, etc.) are saved locally, then queued as an offline action to update Appwrite once online.

D. Moderation & Abuse Reporting (Queued Offline):

- Safety reports are queued (REPORT_GROUP, REPORT_MESSAGE) to prevent them from being lost.

Qbase Network Connectivity & Offline Resilience Audit

E. AI Brain & Quiz Generation (Gated Offline):

- Strictly gated (disabled). AI features require immediate bidirectional connections, so they are deactivated offline.

F. File Transfers & Study Collection Sharing (Gated Offline):

- Strictly gated (disabled). Sending large ZIP binary payloads requires stable connection to upload/download chunks and wrap keys.

Qbase Network Connectivity & Offline Resilience Audit

4. Reconnection & Auto-Recovery Sequence

When NetworkMonitor detects that connection shifts from offline to online, the app initiates an automatic recovery sequence:

1. Recover Messaging: flushes all PENDING messages (encrypting and sending to Appwrite).
2. Recover Admin/Profile updates: flushes all pending offline actions in the Universal Action Queue sequentially.
3. Restores real-time listeners: syncs user chats and starts Appwrite realtime message subscription.

Reconnection Sequence Diagram:

Qbase Network Connectivity & Offline Resilience Audit

